

CircuitDigitizer: Recovering Simulatable Netlists from Hand-Drawn Circuit Schematics with a Degree-Budget Topology Join

Bosco Chanam^{a,*}, Chris Dcosta^a, Pranavesh Kumar Talupuri^b

^a*Symbiosis Centre for Applied Artificial Intelligence, Department of Computer Science and Engineering, Symbiosis Institute of Technology, Pune, India*

^b*Thomas Lord Department of Computer Science, University of Southern California, Los Angeles, California, United States*

Abstract

Converting a photographed hand-drawn circuit schematic into a netlist that an analog simulator can actually run is hard for a reason that is easy to overlook: even when components and wire strokes are detected well, the recovered circuit is almost always *fragmented*. Faint or occluded strokes leave component terminals unconnected, so the netlist breaks into electrically isolated pieces and cannot be simulated. CircuitDigitizer is a transparent, stage-by-stage pipeline that addresses this end to end. A trained oriented-bounding-box detector (YOLO26M-OBB, 16 classes, 88.5% mAP50) localizes components and guides occlusion and region-of-interest cropping; a deterministic Sauvola-based computer-vision stage extracts wire segments (point-based wire F1 = 0.9755 on 134 manually verified images); and the central contribution, the *Degree-Budget Topology Join*, converts detected wires and components into a connected netlist. The join first builds an endpoint graph and then treats every still-floating component terminal — the signature of a wire the detector dropped — as a demand to be satisfied by a reach-bounded minimum-cost matching, capped at one new edge per terminal and guarded against self-loops. This recovers dropped connections while the bounded edge budget structurally prevents the over-merging that shorts dense layouts. Circuit connectivity rises from 68.8% (a strong endpoint-graph baseline) to

*Corresponding author

Email addresses: bosco.chanam.btech2021@sitpune.edu.in (Bosco Chanam), talupuri@usc.edu (Pranavesh Kumar Talupuri)

81.9% with no per-image regressions, and the resulting netlists are emitted as SPICE for DC operating-point simulation. Because real schematics have no net-level ground truth, the join is also evaluated on a synthetic harness that authors known circuits, lays them out as detector-style coordinate maps, injects detector-style error, and scores the real join and simulator against the authored truth; the same harness was used to discover the method. As a baseline, a strong vision-language model (Mimo 2.5) prompted to read the same schematics end to end missed 38–94% of the topology, underscoring that explicit, inspectable topology reasoning remains necessary. No wire-specific training is required for the extraction or join stages.

Keywords: Circuit diagram digitization, Netlist extraction, Topology join, Hand-drawn schematics, SPICE simulation, Oriented object detection, Document image analysis

1. Specifications table

Table 1: Specifications table

Subject area	Computer Science
More specific subject area	Computer vision, document image analysis, circuit diagram digitization, electronic design automation
Name of the method	CircuitDigitizer, with the Degree-Budget Topology Join as the core netlist-recovery stage
Name and reference of original method	Endpoint-graph wire joining (repository baseline); Sauvola adaptive document binarization [2]; oriented-bounding-box detection with Ultralytics YOLO [22]
Resource availability	Pipeline repository: https://github.com/boscochanam/circuit-digitization . Component detector (YOLO26M-OBB, 16-class): https://huggingface.co/boscochanam/circuit-component-detector . Training corpus: CGHD-1152 [1]. Evaluation set: 134 manually verified hand-drawn images with paired component and wire annotations, provided by the authors.
Main validation metric	Primary: point-based wire-segment F1 (20 px endpoint tolerance), 0.9755 on 134 images. Secondary: circuit connectivity (fraction of components on a multi-component net), 81.9% for the Degree-Budget Topology Join versus 68.8% for the endpoint-graph baseline.

2. Background

Converting a hand-drawn circuit schematic into a form that an analog simulator can run requires more than recognizing the drawing: it requires recovering the *electrical topology* — which component terminals are connected

to which. Component and symbol recognition for engineering and circuit diagrams has advanced considerably through detector-based pipelines, graph-enhanced postprocessing, and domain datasets [1, 12, 21, 13, 14, 15, 16, 17, 18]. Strong results are routinely reported for symbol detection, yet the conversion all the way to a connected, simulatable netlist remains the harder and less solved step, because wire recovery is sensitive to clutter, rotation, scan noise, and faint traces, and because a single missed stroke leaves the recovered circuit fragmented and therefore unsimulatable [20, 21, 17].

Wire recovery rests on classical document-analysis tools. Local adaptive thresholding (Niblack, Sauvola, and contrast- or background-based refinements) preserves thin foreground strokes on degraded documents far better than global thresholding [3, 4, 2, 5, 6, 7], and skeletonization and line-segment detection are mature tools for tracing elongated structure and preserving topology [8, 9, 10, 11]. These remain decisive when the objective is recovering sparse geometric structure from weak, fragmented strokes rather than merely segmenting foreground.

Two gaps motivate the present method. First, recognition alone is insufficient: work on piping, single-line, and circuit diagrams consistently finds that symbol detection must be followed by domain-aware reasoning over lines, ports, and connectivity to yield usable structure [14, 15, 16, 17, 12]. General-purpose vision-language models are an attractive shortcut, but, as our vision-language baseline in the Method validation shows, a strong VLM prompted to read a schematic end to end recovers the components and gist while missing a large fraction of the electrical topology — exactly the part that determines whether the circuit simulates. Explicit, inspectable topology reasoning is therefore still needed. Second, the topology step is hard to *evaluate*: hand-drawn corpora annotate component boxes and wire strokes, but not which terminals are electrically common, so there is no net-level ground truth against which to measure or tune a join. This work addresses both: a degree-budget topology join that explicitly recovers dropped connections, and a synthetic evaluation harness that supplies net-level ground truth by construction and was used to select the join.

The manuscript is written as a MethodsX article because the principal contribution is methodological: a reproducible, component-guided pipeline whose core stage – the Degree-Budget Topology Join – is documented in enough detail to be inspected, reused, and re-evaluated, together with the synthetic protocol used to validate it.

3. Method details

CircuitDigitizer is organized as a transparent, stage-by-stage pipeline that takes a photographed hand-drawn schematic and emits a SPICE netlist. Components are localized by a trained oriented detector and used to guide preprocessing; wire segments are extracted by a deterministic computer-vision stage; the *Degree-Budget Topology Join* converts detected wires and components into a connected netlist; and the netlist is exported as SPICE. The full architecture is shown in Fig. 1. The join is the central methodological contribution and is described in its own subsection. Every stage is inspectable in isolation, and no wire-specific training is required for the extraction or join stages.

Datasets and evaluation set

Component priors and the component detector were obtained from the CGHD-1152 hand-drawn circuit corpus, which provides oriented component annotations [1]. The pipeline is evaluated on a set of 134 hand-drawn schematic photographs for which both component annotations and manually verified wire annotations are available (3,524 wire segments in total). Wire extraction is scored with the repository metric: a detected segment is a true positive when its endpoints fall within a 20 px mean tolerance of the nearest ground-truth segment, and precision, recall, and F1 are reported. Because real schematics provide no *net-level* ground truth — there is no annotation of which terminals are electrically common — the join is evaluated both by a structural connectivity metric on the real set and by a synthetic ground-truth harness (described below) that supplies net-level truth by construction.

Component detection

Components are localized with an oriented-bounding-box detector (YOLO26M-OBB [22]) trained on CGHD-1152. The original 61 symbol classes were merged into 16 functional classes; this merging was the single most important design choice and raised mean average precision from roughly 50% to the final 88.5% mAP50 (78.3% mAP50-95). The detector is the single source of truth for component identity and location, replacing earlier reliance on pre-trained third-party labels with non-standard class indices. Detection runs in approximately one second per image on CPU.

Component-guided preprocessing

Each detected component polygon is occluded by filling it with the local median intensity estimated from a margin around the polygon (15% of the bounding-box size, minimum 5 px) [19]. This suppresses symbol strokes and printed text that would otherwise be converted into spurious wire evidence, while leaving junction points intact. The image is then cropped to the union of all component boxes plus 10 px of padding, which removes scanner borders and paper edges; the crop offset is tracked so that extracted endpoints can be mapped back to image coordinates.

Wire extraction

Wire evidence is produced by Sauvola adaptive thresholding [2] in a 67×67 window with $k = 0.285$, chosen because local adaptive thresholding preserves thin foreground strokes on degraded document content far better than global methods. The binary mask is closed with a 3×3 elliptical kernel, labeled into connected components (minimum area 28 px), and each component is reduced to a line segment whose endpoints are obtained by principal-component analysis rather than by extremal pixels. Near-duplicate segments are removed by an overlap test (angle 12° , distance 8 px), and an anchor filter (endpoint distance 16 px, link distance 8 px) discards floating endpoints that do not reach a component or another wire. No global line-merging or length-filtering stage is used; both were found to destroy true positives.

The Degree-Budget Topology Join

The join converts the extracted wires and detected components into electrical nodes. It proceeds in two phases: an endpoint-graph base join, followed by a degree-budget completion that recovers the connections lost when the detector drops or truncates a wire.

Phase 1 — endpoint-graph base.. Component pins are derived from each oriented box (two terminals on the short-axis midpoints for two-terminal parts, class-specific layouts otherwise), and wires are extended 12 px along their direction to close occlusion gaps. Wire endpoints and pins are treated as nodes of a single graph and merged by union-find under three edge types: a wire body links its own two endpoints; two endpoints within a scale-relative tolerance τ_{join} are joined (fragments, corners); and an endpoint binds the pin it points at, assigned component-first — the endpoint is attached to the

nearest component by bounding-box proximity and then routed to the correct pin by geometry, which handles endpoints that land inside a component box but far from its pin. A dead-end rescue gives a wire anchored on one pin a longer directional reach toward a pin on a different component, recovering strokes the detector cut short. Tolerances are expressed as multiples of the median component size so a single rule spans the wide scale range of hand drawings.

Phase 2 — degree-budget completion.. After the base join, a non-inert component terminal whose net touches only its own component is *floating*: it is the signature of a wire the detector missed entirely, which the base join cannot recover because no stroke connects it. Ground (gnd), text, junction, and other single-terminal or non-conductive classes are excluded from this notion. Each floating terminal is treated as a unit demand to be satisfied by at most one new edge. Candidate completion edges are generated to pins on *other* components within a reach of $2.5 \tau_{\text{pin}}$, excluding pins already on the terminal’s net; the cost of an edge prefers a geometric witness (an actual wire whose two ends lie near the two pins) and falls back to pure Euclidean distance when no witness exists, so that genuinely dropped wires can still be completed. The resulting bipartite problem — floating terminals against candidate target pins, each target offering up to three slots so that legitimate multi-terminal nets are not starved — is solved as a minimum-cost assignment with the Hungarian algorithm [23], with a per-terminal opt-out so that a terminal with no good target is left unconnected rather than forced. The chosen matches are applied with a *self-loop guard*: a match that would merge two nets already sharing a component is skipped, so a two-terminal part can never be shorted onto a single net.

This degree budget is what makes the method safe. Because every floating terminal contributes at most one edge and the guard forbids self-loops, the completion recovers dropped connections without the all-to-all over-merging that shorts dense or bridge-like layouts. Base-join wire indices are carried onto the final nodes so that downstream simulation and current visualization retain wire-level provenance; completion edges are inferred and carry no wire, exactly as a manually drawn connection would.

SPICE generation

Each electrical node becomes a SPICE [24] node and each simulatable component (resistor, capacitor, inductor, diode, voltage source, and similar)

becomes a device line; a ground node is mapped to node 0 when present. The emitted deck is run through `ngspice` for a DC operating-point analysis, yielding node voltages and device currents that are surfaced in the interface.

Synthetic ground-truth evaluation

Because the real set has no net-level ground truth, the join is also evaluated on a synthetic harness that runs the loop in the opposite direction. A small catalog of circuits with known netlists is authored; each is laid out as a coordinate map of component boxes and wire segments of the same form the detector produces; detector-style error (endpoint jitter, cut-short truncation, dropped wires, wrong-pin snaps, and anchor displacement of 30–80 px) is injected at increasing severities; and the *real* join and SPICE generator are run on the perturbed map and scored against the authored truth. Two signals are reported: a component-connectivity F1 against the authored nets, and whether the recovered circuit simulates to the authored operating point. The harness supplies net-level truth by construction, doubles as a regression test (the clean, error-free map must round-trip to $F1 = 1.0$), and — by searching over candidate join algorithms against this truth — was the means by which the Degree-Budget Topology Join was discovered and selected over alternatives. Its one caveat is that the injected error model is a first-pass approximation rather than a calibrated model of the real detector’s failure distribution; it is therefore used as a robustness and regression signal, with the real benchmark remaining the decisive setting.

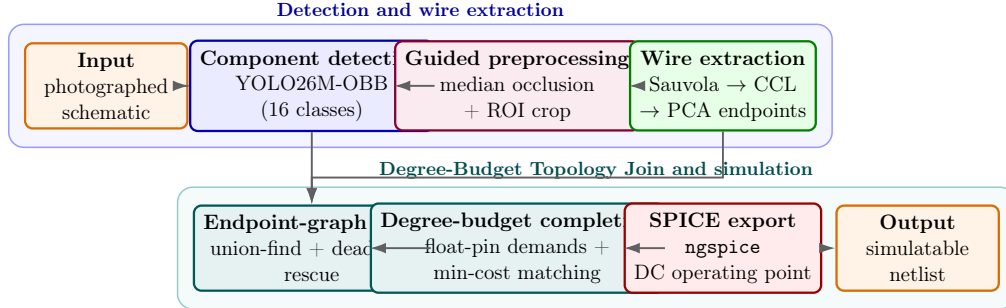


Figure 1: CircuitDigitizer pipeline. A trained oriented detector localizes components and guides occlusion and cropping; a deterministic Sauvola-based stage extracts wire segments; the Degree-Budget Topology Join builds an endpoint graph and then completes floating component terminals with a reach-bounded, self-loop-guarded minimum-cost matching; and the connected netlist is exported as SPICE.

4. Method validation

The pipeline is validated at three levels: wire extraction against annotated wire segments, the Degree-Budget Topology Join against both a real connectivity metric and synthetic net-level ground truth, and the emitted netlists against a SPICE simulator. An end-to-end vision-language baseline is reported for context.

Component and wire extraction

The trained oriented detector reached 88.5% mAP50 (78.3% mAP50-95, precision 95.6%, recall 88.6%) on the held-out split, with every functional class detected; the visually hardest class is crossover (70.7% recall), where two crossing wires are genuinely ambiguous. On the 134-image evaluation set the deterministic wire stage achieved a point-based wire F1 of 0.9755 (precision 0.9729, recall 0.9781) in its best configuration. The distribution is strongly skewed toward clean recovery: the median per-image F1 is 1.000, 117 of 134 images (87%) score $F1 \geq 0.90$, and only 4 images (3%) fall below 0.50 — dense layouts where large component occlusion removes wire endpoints.

A thresholding study (Table 2) substituted other binarization families into the same downstream extraction and evaluation logic. Sauvola thresholding dominated every alternative: adaptive Gaussian reached $F1 = 0.928$, global Otsu 0.828, and triangle thresholding 0.795. Skeleton-based extraction lost substantial recall by breaking thin wires, and threshold-fusion variants added nothing over Sauvola alone, confirming that the strong result rests on conservative local thresholding rather than on any single fusion trick. A parameter sweep over k , window size, link distance, and deduplication angle moved F1 only marginally, indicating that the extraction stage is robust.

Table 2: Wire-extraction thresholding comparison on the 134-image benchmark, all run through the identical preprocessing, extraction, and evaluation logic.

Thresholding family (best setting)	Wire F1
Sauvola ($k = 0.285$, $w = 67$, anchor dist. 16)	0.9755
Sauvola (baseline, anchor dist. 12)	0.9730
Adaptive Gaussian	0.928
Global Otsu	0.828
Triangle	0.795

Topology join

The central result is the connectivity gain from the Degree-Budget Topology Join. Connectivity is measured as the fraction of components placed on a net shared with at least one other component; a fragmented circuit scores low because its pieces are electrically isolated and cannot be simulated. Relative to the endpoint-graph base join, the degree-budget completion raises connectivity from 68.8% to 81.9% (Table 3) and does so without a single per-image regression: every image is recovered at least as well as the base join. The connectivity gain comes from a sharp reduction in floating component terminals: the completion reconnects terminals the detector had left dangling. The one-edge-per-terminal degree budget and the self-loop guard bound the cost of this recovery, and on the synthetic net-level ground truth (below) the join’s precision does not fall, which indicates the recovered connections are genuine rather than over-merges. On the densest real layouts the completion can still introduce a small number of two-terminal self-loops; this residual cost is discussed in the Limitations.

Table 3: Topology-join comparison. Connectivity is the fraction of components on a multi-component net; self-loops count two-terminal parts shorted onto one net; regressions count images where connectivity falls below the base join.

Join strategy		Connectivity (%)	Regressions	Synthetic F1	Clean F1
Endpoint-graph	base	68.8	—	0.944	1.000
(<i>graph_rescue</i>)					
Degree-Budget	Topol-	81.9	0	0.969	1.000
ogy Join					

Because the real connectivity metric is structural and cannot certify that a recovered connection is the *correct* one, the join was also scored against net-level ground truth on the synthetic harness. On a catalog of authored circuits spanning series, parallel, bridge, multi-source, and adjacent-loop topologies, the clean (error-free) coordinate maps round-trip to a component-connectivity F1 of 1.000 for the join, confirming the layouts and the join logic are sound. Under the injected detector-style error, the degree-budget join attains a mean across-severity F1 of 0.969 versus 0.944 for the endpoint-graph base; it wins at every severity, and its advantage is concentrated in the dropped-wire regime, where the completion recovers connections that no purely geometric join can. This is consistent with the real connectivity

gain and indicates the gain reflects recovered true connections rather than over-merging.

SPICE simulation

Netlists emitted by the pipeline simulate under `ngspice`: DC operating-point analysis returns node voltages and device currents that obey Kirchhoff’s current law on the validation circuits. The synthetic harness provides a stronger check — it supplies an analytic operating point for each authored circuit — and the simulated source currents match the authored values for every clean circuit (for example, an authored voltage divider returns its exact divider current). The same harness shows the qualitative payoff of the join: a circuit that is fragmented by a dropped wire simulates as dead, whereas after degree-budget completion it simulates to the correct operating point.

Vision-language baseline

To test whether an off-the-shelf model could replace the structured pipeline, a strong vision-language model (Mimo 2.5) was prompted to read the schematics directly and emit structured connectivity. On 10 representative images it missed between 38% and 94% of the topology, producing plausible-looking but electrically wrong netlists. The failure mode is informative: the model recognizes components and the gist of the drawing but does not reliably track which terminals are electrically common, which is exactly the problem the Degree-Budget Topology Join solves explicitly and inspectably. (We note that the repository’s general VLM utility defaults to a different model for paper-type classification; the end-to-end topology baseline reported here used Mimo 2.5.)

Representative pipeline stages are shown in Fig. 2: for each example the input image, the cropped region after component and text suppression, the Sauvola wire mask, and the final prediction overlay are shown, so that the effect of each stage can be inspected directly.

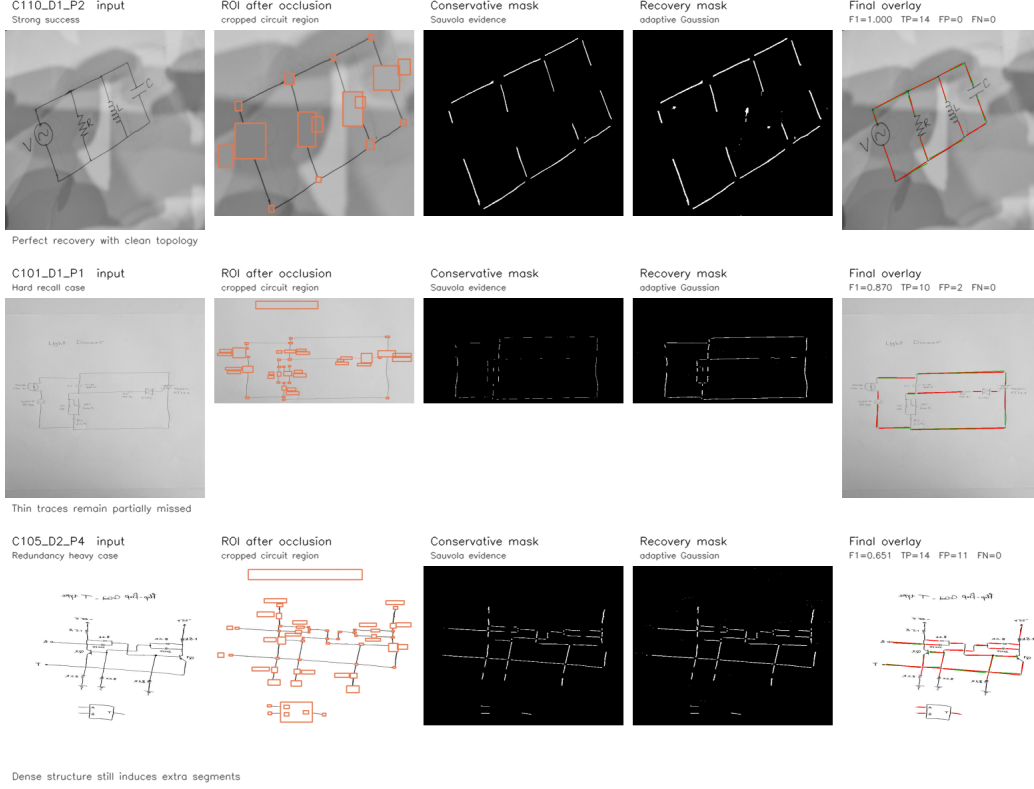


Figure 2: Representative pipeline stages. For each example the input image, the cropped region after component and text suppression, the Sauvola wire mask, and the final prediction overlay are shown. Green segments denote ground truth and red segments denote predicted wire segments.

5. Limitations

Several limitations should be acknowledged.

First, the join is evaluated on real images only by a *structural* connectivity metric. Hand-drawn corpora annotate component boxes and wire strokes but not which terminals are electrically common, so there is no net-level ground truth on real images against which to certify that a recovered connection is the *correct* one. The synthetic harness supplies net-level truth by construction and shows the completion does not reduce precision there, but a manually net-annotated real benchmark remains important future work.

Second, the synthetic harness is only as faithful as its error model. The injected perturbations (jitter, cut-short, dropped wires, wrong-pin snaps,

anchor displacement) are a first-pass approximation, not a model calibrated to the real detector’s measured failure distribution. Synthetic join scores are therefore used as robustness and regression signals, with the real benchmark remaining the decisive setting; calibrating the error model to the detector is future work.

Third, the recovery has a residual cost. On the densest real layouts the degree-budget completion can introduce a small number of two-terminal self-loops (a component shorted onto one net). The one-edge-per-terminal budget and the self-loop guard bound this effect rather than eliminating it, and it is the price of the large reduction in floating terminals that drives the connectivity gain.

Fourth, the hardest detection failures remain recall-driven and concentrated in dense drawings: four of the 134 evaluation images score wire F1 below 0.50, where large component occlusion removes nearby wire endpoints, and crossover is the weakest component class (70.7% recall) because crossing wires are visually ambiguous. The pipeline also remains dependent on accurate component priors, since occlusion and region-of-interest selection are driven by the detector.

Fifth, the evaluation, although carefully annotated, draws from a single corpus of 134 images; generalization across further drawing styles and acquisition conditions should be validated. Finally, only components with a known device model (resistor, capacitor, inductor, voltage source, diode, transistor) are emitted to SPICE; integrated circuits, op-amps, and similar parts are detected and placed in the topology but skipped at the simulation stage, so circuits dominated by such parts are only partially simulatable.

6. CRediT author statement

Bosco Chanam: Conceptualization, methodology, supervision, writing review and editing.

Chris Dcosta: Data curation, software, validation, writing review and editing.

Pranavesh Kumar Talupuri: Methodology, software, formal analysis, visualization, writing of the revised manuscript.

7. Data availability

The pipeline repository is available at <https://github.com/boscochanam/circuit-digitization>. The trained 16-class component detector (YOLO26M-

OBB) is released at <https://huggingface.co/boscochanam/circuit-component-detector> and was trained on the public CGHD-1152 corpus [1]. The evaluation set of 134 manually verified hand-drawn images, with paired component and wire annotations (3,524 wire segments), is maintained by the authors. The synthetic ground-truth circuit catalog and the evaluation harness are part of the repository. Benchmark outputs, figure assets, and manuscript source files are contained in the project workspace.

8. Acknowledgments

Institutional support from the Symbiosis Centre for Applied Artificial Intelligence and the University of Southern California is gratefully acknowledged.

9. Declaration of interests

The authors declare that no known competing financial interests or personal relationships could have appeared to influence the work reported in this paper.

Appendix A. Per image benchmark results

The per-image wire-extraction results for the 134-image benchmark are listed in Table A.4. These values are generated directly from the repository benchmark output and are not entered manually.

Table A.4: Per-image wire-extraction results on the 134-image benchmark.

Image	F1	TP	FP	FN	Red	Tags
C100_D1_P1	0.889	24	1	3	2	–
C100_D1_P3	0.704	19	7	5	4	fp_heavy
C100_D2_P4	0.810	17	3	0	5	redundancy_heavy
C101_D1_P1	0.870	10	2	0	1	–
C102_D1_P2	0.485	8	11	0	6	fp_heavy, redundancy_heavy
C103_D2_P1	0.737	7	1	0	4	redundancy_heavy
C103_D2_P4	0.941	8	0	0	1	–
C104_D1_P3	0.615	8	5	0	5	fp_heavy, redundancy_heavy
C104_D1_P4	0.857	9	1	0	2	–
C104_D2_P2	1.000	13	0	0	0	–
C104_D2_P3	1.000	13	0	0	0	–
C104_D2_P4	0.917	11	2	0	0	–
C105_D1_P1	0.800	16	6	1	1	fp_heavy
C105_D1_P4	0.681	16	9	0	6	fp_heavy, redundancy_heavy
C105_D2_P4	0.651	14	11	0	4	fp_heavy, redundancy_heavy
C109_D2_P3	1.000	7	0	0	0	–
C110_D2_P3	0.714	5	1	3	0	–
C110_D1_P2	1.000	14	0	0	0	–
C111_D1_P1	0.957	11	1	0	0	–
C112_D1_P1	0.870	20	0	5	1	–
C113_D2_P3	1.000	7	0	0	0	–
C115_D2_P3	1.000	7	0	0	0	–
C127_D2_P1	0.927	19	2	0	1	–
C128_D2_P1	0.960	36	0	3	0	–
C128_D2_P4	0.884	38	3	5	2	–
C129_D1_P1	0.973	18	0	1	0	–
C129_D2_P1	0.947	18	0	1	1	–
C12_D1_P1	0.830	39	4	11	1	–
C12_D2_P2	0.884	42	0	11	0	–
C130_D1_P1	0.930	20	3	0	0	–
C131_D1_P1	0.847	25	4	4	1	–
C133_D1_P2	1.000	10	0	0	0	–
C133_D1_P3	1.000	10	0	0	0	–
C134_D2_P1	1.000	8	0	0	0	–
C134_D2_P2	1.000	8	0	0	0	–
C134_D2_P3	1.000	8	0	0	0	–
C134_D2_P4	1.000	8	0	0	0	–
C136_D1_P1	1.000	7	0	0	0	–
C136_D1_P2	1.000	7	0	0	0	–
C136_D1_P3	0.933	7	0	1	0	–
C136_D1_P4	1.000	7	0	0	0	–
C136_D2_P1	0.800	6	1	2	0	–
C136_D2_P2	1.000	7	0	0	0	–
C136_D2_P3	1.000	7	0	0	0	–
C137_D1_P1	1.000	12	0	0	0	–
C137_D1_P2	0.960	12	0	1	0	–
C137_D1_P3	0.960	12	0	1	0	–
C138_D1_P3	0.909	15	1	2	0	–
C138_D2_P1	1.000	16	0	0	0	–
C138_D2_P2	0.970	16	1	0	0	–
C157_D1_P1	0.958	23	0	2	0	–
C157_D1_P2	0.958	23	0	2	0	–
C157_D1_P3	0.958	23	0	2	0	–
C157_D2_P1	0.979	23	0	1	0	–
C157_D2_P3	0.958	23	0	1	1	–
C158_D1_P1	0.878	18	0	5	0	–
C15_D2_P2	0.935	29	0	3	15	–
C163_D2_P4	0.941	24	0	2	1	–
C165_D1_P2	0.857	9	0	3	0	–
C167_D2_P1	1.000	10	0	0	0	–
C182_D2_P2	0.857	60	2	14	4	–
C187_D1_P3	0.857	33	1	7	3	–
C192_D1_P1	0.848	42	3	8	4	–
C193_D2_P2	0.855	50	0	16	1	–
C199_D1_P3	0.941	32	0	4	0	–
C19_D1_P2	0.960	36	0	3	0	–

References

- [1] F. Thoma, J. Bayer, Y. Li, A. Dengel, A public ground truth dataset for handwritten circuit diagram images, in: International Conference on Document Analysis and Recognition, Springer, 2021.
- [2] J. Sauvola, M. Pietikäinen, Adaptive document image binarization, *Pattern Recognition* 33 (2) (2000) 225–236.
- [3] N. Otsu, A threshold selection method from gray-level histograms, *IEEE Transactions on Systems, Man, and Cybernetics* 9 (1) (1979) 62–66.
- [4] W. Niblack, *An Introduction to Digital Image Processing*, Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [5] B. Gatos, I. Pratikakis, S. J. Perantonis, Adaptive degraded document image binarization, *Pattern Recognition* 39 (3) (2006) 317–327.
- [6] F. Shafait, D. Keysers, T. Breuel, Efficient implementation of local adaptive thresholding techniques using integral images, in: *Document Recognition and Retrieval XV*, Vol. 6815, SPIE, 2008, 681510.
- [7] B. Su, S. Lu, C. L. Tan, Robust document image binarization technique for degraded document images, *IEEE Transactions on Image Processing* 22 (4) (2013) 1408–1417.
- [8] T. Y. Zhang, C. Y. Suen, A fast parallel algorithm for thinning digital patterns, *Communications of the ACM* 27 (3) (1984) 236–239.
- [9] L. Lam, S.-W. Lee, C. Y. Suen, Thinning methodologies: a comprehensive survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14 (9) (1992) 869–885.
- [10] R. O. Duda, P. E. Hart, Use of the Hough transformation to detect lines and curves in pictures, *Communications of the ACM* 15 (1) (1972) 11–15.
- [11] R. Grompone von Gioi, J. Jakubowicz, J.-M. Morel, G. Randall, LSD: a fast line segment detector with a false detection control, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32 (4) (2010) 722–732.

- [12] R. R. Rachala, M. R. Panicker, Hand drawn electrical circuit recognition using object detection and node recognition, *SN Computer Science* 3 (3) (2022) 244.
- [13] B. Schäfer, M. Keuper, H. Stuckenschmidt, Arrow R-CNN for handwritten diagram recognition, *International Journal on Document Analysis and Recognition* 24 (1–2) (2021) 3–17.
- [14] E. Elyan, L. Jamieson, A. Ali-Gombe, Deep learning for symbols detection and classification in engineering drawings, *Neural Networks* 129 (2020) 91–102.
- [15] D.-Y. Yun, S.-K. Seo, U. Zahid, C.-J. Lee, Deep neural network for automatic image recognition of engineering diagrams, *Applied Sciences* 10 (11) (2020) 4005.
- [16] L. Yang, J. Wang, J. Zhang, H. Li, K. Wang, C. Yang, D. Shi, Practical single-line diagram recognition based on digital image processing and deep vision models, *Expert Systems with Applications* 238 (2024) 122389.
- [17] D. Hemker, J. Maalouly, H. Mathis, R. Klos, E. Ramanan, From schematics to netlists: electrical circuit analysis using deep-learning methods, *Advances in Radio Science* 22 (2024) 61–75.
- [18] L. Jamieson, C. F. Moreno-García, E. Elyan, A review of deep learning methods for digitisation of complex documents and engineering diagrams, *Artificial Intelligence Review* 57 (2024) 136.
- [19] G. Bradski, The OpenCV Library, *Dr. Dobb’s Journal of Software Tools* (2000).
- [20] M. K. A. Rabby et al., A survey on hand drawn circuit diagram digitization: techniques, datasets and challenges, *IEEE Access* (2019).
- [21] Y. Shen et al., Automatic circuit schematic recognition: A review, *Engineering Applications of Artificial Intelligence* 126 (2023).
- [22] Ultralytics, Ultralytics YOLO documentation, 2024. <https://github.com/ultralytics/ultralytics> (accessed 2026-05-25).

- [23] H. W. Kuhn, The Hungarian method for the assignment problem, Naval Research Logistics Quarterly 2 (1–2) (1955) 83–97.
- [24] L. W. Nagel, D. O. Pederson, SPICE (Simulation Program with Integrated Circuit Emphasis), Technical Report ERL-M382, Electronics Research Laboratory, University of California, Berkeley, 1973.